OS Orange

Name: Pragya Chowdary S

SRN: PES2UG24AM115

SEC: 4B

1a:

```
pragya@pragya-VirtualBox:~/PES2UG24AM115-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
pragya@pragya-VirtualBox:~/PES2UG24AM115-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
test_objects: test_objects.c:80: test_integrity: Assertion `rc == -1' failed.
Aborted (core dumped)
pragya@pragya-VirtualBox:~/PES2UG24AM115-pes-vcs$
```

1b:

```
pragya@pragya-VirtualBox:~/PES2UG24AM115-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
pragya@pragya-VirtualBox:~/PES2UG24AM115-pes-vcs$
```

2a:

```
pragya@pragya-VirtualBox:~/PES2UG24AM115-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2b:

```
pragya@pragya-VirtualBox:~/PES2UG24AM115-pes-vcs$ cat .pes/index
304444 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 0 6 hello.txt
304444 66224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39 0 6 hello.txt
304444 05c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43 0 12 hello.txt
304444 e67ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc 0 8 bye.txt
pragya@pragya-VirtualBox:~/PES2UG24AM115-pes-vcs$ xxd 304444 2cf8d83d9ee29543b34a87727421fdecb7e3f3a
83d337639025de576db9ebb4 0 6 hello.txt
Usage:
       xxd [options] [infile [outfile]]
    or
       xxd -r [-s [-]offset] [-c cols] [-ps] [infile [outfile]]
Options:
    -a          toggle autoskip: A single '*' replaces nul-lines. Default off.
    -b          binary digit dump (incompatible with -ps,-i). Default hex.
    -C          capitalize variable names in C include file style (-i).
    -c cols     format <cols> octets per line. Default 16 (-i: 12, -ps: 30).
    -E          show characters in EBCDIC. Default ASCII.
    -e          little-endian dump (incompatible with -ps,-i,-r).
    -g bytes    number of octets per group in normal output. Default 2 (-e: 4).
    -h          print this summary.
    -i          output in C include file style.
    -l len      stop after <len> octets.
    -n name     set the variable name used in C include output (-i).
    -o off      add <off> to the displayed file position.
    -ps         output in postscript plain hexdump style.
    -r          reverse operation: convert (or patch) hexdump into binary.
    -r -s off   revert with <off> added to file positions found in hexdump.
    -d          show offset in decimal instead of hex.
    -s [+][-]seek  start at <seek> bytes abs. (or +: rel.) infile offset.
    -u          use upper case hex letters.
    -R when     colorize the output; <when> can be 'always', 'auto' or 'never'. Default: 'auto'.
    -v          show version: "xxd 2023-10-25 by Juergen Weigert et al.".
```

3a:

```
pragya@pragya-VirtualBox:~/PES2UG24AM115-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  modified:   file1.txt
  modified:   file2.txt

Untracked files:
  untracked:  test_tree.c
  untracked:  Makefile
  untracked:  object.c
  untracked:  README.md
  untracked:  test_sequence.sh
  untracked:  commit.c
  untracked:  test_objects.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  .gitignore
  untracked:  tree.h
  untracked:  commit.h
  untracked:  index.c
  untracked:  index.h
  untracked:  tree.c
```

3b

```
pragya@pragya-VirtualBox:~/PES2UG24AM115-pes-vcs$ cat .pes/index
304444 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 0 6 file1.txt
304444 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 0 6 file2.txt
```

4a

```
commit a9af6f2081eb0227cae466ecece83e4cbedfa67ae7ede3db487ba2fa6a226258
Author: PES User
Date:   0

    Add farewell
d farewel!

commit 31fb1a475da36ce04e28df16bd5b0059315e40244286d34edd29997d89d230e2
Author: PES User
Date:   0

    Add world


commit e592984d2b5a1bb665fe6de65f183e09c23cdb83e5975ece18493a43f9689902
Author: PES User
Date:   0

    Initial commit
```

4b

```
pragya@pragya-VirtualBox:~/PES2UG24AM115-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/2f/95a7299b62529ed2c0fefdf15f01229602af214e08ec971ca82b1a326d254e
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/refs/heads/main
```

4c

```
pragya@pragya-VirtualBox:~/PES2UG24AM115-pes-vcs$ cat .pes/refs/heads/main
a9af6f2081eb0227cae466ecece83e4cbedfa67ae7ede3db487ba2fa6a226258
pragya@pragya-VirtualBox:~/PES2UG24AM115-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
pragya@pragya-VirtualBox:~/PES2UG24AM115-pes-vcs$
```

Make test-integration

```
--- Second Commit ---
Committed: fb15345cf5d5... Update file.txt

--- Third Commit ---
Committed: bb04acb0ce8b... Add farewell

--- Full History ---
commit bb04acb0ce8bde045d1a42ebcf7258f76d812804dad77196f767d452728552bc
Author: PES User
Date:   0

    Add farewell
d farewel!

commit fb15345cf5d5589111b79767ddd061a95263759dda304e1f899e455f51092969
Author: PES User
Date:   0

    Update file.txt
arewel

commit d9bb8daf53ca6598ab20da88e43e4428324bee340facb31feff996a8b34a159c
Author: PES User
Date:   0

    Initial commit



--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
bb04acb0ce8bde045d1a42ebcf7258f76d812804dad77196f767d452728552bc

--- Object Store ---
Objects created:
8
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/bb/04acb0ce8bde045d1a42ebcf7258f76d812804dad77196f767d452728552bc
.pes/objects/d9/bb8daf53ca6598ab20da88e43e4428324bee340facb31feff996a8b34a159c
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/fb/15345cf5d5589111b79767ddd061a95263759dda304e1f899e455f51092969

=== All integration tests completed ===
```

Q5

A branch in PES-VCS is represented as a file inside .pes/refs/heads/ that stores the hash of the latest commit. The HEAD file points to the current branch.

To implement pes checkout <branch>, the following steps are required:

1. First, verify whether the specified branch exists by checking for the file .pes/refs/heads/<branch>. If it does not exist, the operation should fail with an error.
2. Update the HEAD file so that it points to the new branch. This is done by writing:

ref: refs/heads/<branch>

3. Read the commit hash stored in the branch file and load the corresponding commit object from the object store.
4. From the commit object, retrieve the tree hash representing the snapshot of the project.
5. Reconstruct the working directory by traversing the tree object recursively. For each entry:
    - If it is a blob, recreate the file with its contents.
    - If it is a tree, recreate the directory and process its contents.

This operation is complex because it involves safely modifying the working directory. It must correctly handle file creation, deletion, and overwriting, while ensuring that no important data is lost. Additionally, it must consider uncommitted changes, which can make the operation unsafe if not handled properly.

Q5.2

How to detect a dirty working directory?

A “dirty working directory” refers to a situation where the user has made changes that are not yet committed.

To detect this, the system compares the working directory with the index:

1. The index stores metadata for each staged file, including file size, modification time, and hash.
2. For each file in the index:
    - Compare the current file’s size and modification time with the stored values.
    - If they differ, recompute the file’s hash and compare it with the stored hash.
3. If the hash differs, the file has been modified and is considered “dirty.”

During checkout, if a file is modified in the working directory and also differs between the current branch and the target branch, a conflict occurs. In such cases, the checkout operation must be aborted to prevent loss of uncommitted changes.

Thus, detecting a dirty working directory relies on comparing:

- The index (expected state)
- The working directory (current state)

Q5.3

A detached HEAD occurs when the HEAD file points directly to a commit hash instead of a branch reference.

In this state:

New commits can still be created.

However, these commits are not associated with any branch.

As a result, the new commits are not reachable through any branch and can eventually be lost during garbage collection.

To recover such commits, the user can:

Create a new branch pointing to the commit hash.

Or manually update a branch reference to include the commit.

Thus, while detached HEAD allows temporary exploration of past commits, it is risky because any new work may become unreachable if not explicitly saved.

Q6.1

Over time, the object store may contain objects that are no longer referenced by any branch. These are called unreachable objects.

To identify and remove them, a mark-and-sweep algorithm can be used:

Start from all branch heads located in .pes/refs/heads/.

For each branch:

Load the commit object.

Traverse its parent commits recursively.

For each commit, follow its tree.

For each tree, visit all blobs and subtrees.

Maintain a set of all visited (reachable) object hashes.

After traversal, scan the entire .pes/objects/ directory.

Any object not present in the reachable set is considered unreachable and can be safely deleted.

A hash set is the most suitable data structure for tracking reachable objects, as it allows constant-time lookup and efficient handling of large numbers of objects.

For a repository with 100,000 commits and multiple branches, the traversal may involve visiting hundreds of thousands of objects, including commits, trees, and blobs.

Q6.2

Garbage collection is unsafe when executed concurrently with a commit operation due to possible race conditions.

A typical race condition scenario is:

A commit operation creates new objects (blobs, trees, commit) but has not yet updated the branch reference.

At the same time, garbage collection runs and scans for reachable objects.

Since the new commit is not yet referenced by any branch, GC considers its objects unreachable and deletes them.

The commit operation then completes and updates the branch to point to the now-deleted objects.

This results in a corrupted repository, where references point to non-existent data.

To avoid this, real systems like Git use several techniques:

Locking mechanisms to prevent concurrent modifications.

Atomic updates to ensure references are updated safely.

Temporary protection of newly created objects.

Delayed garbage collection, ensuring that only old unreachable objects are removed.

Thus, garbage collection must be carefully coordinated with commit operations to maintain repository integrity.